4-DAY SPRINT · ESP32

Configurable TinyML Framework 4-Day Build Guide

What to build, in what order, and how to test each stage

Start: Wednesday, September 30, 2026

Finish: end of Day 4 — Sunday, October 4, 2026

Hardware for this sprint: ESP32 dev board + USB cable

Later (not in these 4 days): port to NXP FRDM-MCXN236 (Cortex-M33)

How to use this guide

The project is a **pipeline**. Each stage below has three parts: **what to build**, **how to test it**, and a **Go/No-Go gate**. Do not start a stage until the previous stage's gate has passed. If a gate fails, check the "If it fails" box and the Debugging Map (last page) before moving on.

```
DATASET ↓ TRAIN (FP32) ↓ QUANTIZE (INT8 + bit-exact simulator)
   ↓ EXPORT (model.json + model_data.c/.h + golden vectors)
   ↓ BUILD C RUNTIME (static buffers + dispatcher + kernels)
   ↓ RUN ON ESP32 ↓ VERIFY BIT-EXACT vs golden vectors
   ↓ BENCHMARK ↓ RESULTS TABLE + DEMO
```

The one rule above all: verify bit-exactness at every layer. Every stage's output must match a trusted reference before it feeds the next stage. This is what makes a 4-day build possible — you never debug two unknowns at once.

STAGE 0 Day 1 Morning — Setup & Contracts (2–3 h)

Everything downstream depends on agreeing data formats *before* writing kernels or runtime code. Do this first.

0.1 — Install and verify the toolchains

Tool	What it is for	Verify it works
Python 3.10+ with NumPy, PyTorch (CPU is fine)	Training, quantization, simulator, exporters	`python -c "import torch, numpy; print(torch.__version__)"` prints a version
ESP-IDF v5.x	ESP32 compiler + build system + flash/monitor	`idf.py --version` works; build and flash the `hello_world` example, see output in `idf.py monitor`
CMake 3.16+ and a host C compiler	Fast host build for kernel tests	`cmake --version` and `gcc --version` (or `clang`) print versions
Git + a GitHub repo	Version control; push at every gate	You can push a commit from the project folder

> **If it fails:** the most common Day-1 blocker is ESP-IDF. If `hello_world` will not build/flash after ~2 h of effort, continue with the host build (Stage 4 works on host too) and fix ESP-IDF in parallel. Do not burn the day on it.

0.2 — Create the repository structure

```
tinyml-mcxn236/
├── tools/              # Python: train, quantize, exporter, simulator, golden-vector generator
├── firmware/
│   └── src/
│       ├── runtime/    # dispatcher (walks the model, calls kernels in order)
│       ├── kernels/    # fc_int8, conv1d_int8, relu, softmax, requantize
│       ├── memory/     # static arena (no malloc)
│       ├── platform/   # HAL: cycle counter, timer, print  (esp32 + host backends)
│       └── benchmark/  # timing harness
├── models/             # exported model.json, model_data.c/.h, golden vectors
├── tests/              # host + on-target golden-vector tests
└── docs/               # the three contract documents live here
```

0.3 — Freeze the three contracts (write them in docs/, commit them)

Contract	Must define
1. Requantize spec	INT8 weights + activations, **int32 accumulators**; requantize by fixed-point multiplier + shift; **round-half-to-even**; saturate to [−128, +127]; per-tensor scales; symmetric weights (zp = 0), asymmetric activations; the exact softmax algorithm (exp LUT or shift-based) — decide now, C will copy it exactly.
2. model.json schema	Ordered layer list: type (fc / conv1d / relu / softmax / flatten), input + output shapes, weight/bias tensor IDs into model_data.c, quant scales and zero-points per tensor, pad/stride/kernel-size for conv1d, total arena size.
3. Golden-vector format	JSON: one file per layer + one per full model. Fields: input (int8 array), expected output (int8 array), scales, zero-points, tolerance (0 = bit-exact). Generated by the Python simulator, consumed by C tests on host and ESP32.

> **GATE 0 — end of Day 1 morning:** hello_world builds and flashes on the ESP32 · host CMake builds a hello test · the three contract docs are committed · repo pushed. Only now start Stage 1.

STAGE 1 Day 1 — Train the Models (FP32)

Two small models. **Do not chase high accuracy** — the sprint proves the framework, not the model. Keep training under ~30 min per model.

1.1 — Pick the dataset (decide today)

Option	Feeds	Notes
A. Synthetic sensor/gesture data (recommended)	MLP + 1D-CNN	NumPy-generated signals (e.g. sine / square / noise classes, or 3-axis synthetic gestures). Zero download time, fully controlled, easy class separation → accuracy ~99%.
B. Google Speech Commands subset (yes/no/up/down)	1D-CNN	Real keyword-spotting case study; download + preprocessing costs time. Use only if you already have it locally.
C. UCI HAR smartphone dataset	1D-CNN	Real sensor data, small download, moderate difficulty.

1.2 — Train (small on purpose)

Model	Architecture	INT8 size (approx)
MLP	Flatten → FC 128 → ReLU → FC 64 → ReLU → FC n_classes	~15 KB
1D-CNN	Conv1D(16, k=5) → ReLU → MaxPool → Conv1D(32, k=3) → ReLU → MaxPool → Flatten → FC n_classes	~25 KB

Adam, cross-entropy, 10–20 epochs. Save the **train/val/test split to disk** — the same split must be reused later for the quantized-accuracy test (never re-split). Also save normalization constants (mean/std per feature): the C side must apply *the same* normalization as training.

> **GATE 1 — end of Day 1:** both models trained · test accuracy printed (expect ≥ 95% synthetic, ≥ 85% real data)¨FP32 weights + normalization constants saved · one command retrains both from scratch.

> **If it fails:** low accuracy → check normalization and class balance; still low → make the dataset easier. Sprint scope beats model quality.

STAGE 2 Day 2 Morning — Quantize (INT8) + Bit-Exact Simulator

This is the heart of the project. The NumPy simulator you build here becomes the **golden reference that every C kernel is tested against**.

2.1 — Post-training quantization (PTQ)

Step	What to do
1. Collect activation ranges	Run ~200 calibration samples through the FP32 model; record per-tensor min/max of every layer output (or use fixed ranges for ReLU nets).
2. Compute quant parameters	`scale = (max - min) / 255` ; `zero_point = round(-min / scale) - 128` (int8). Weights: symmetric, zp = 0.
3. Quantize weights	`q = clip(round(w / scale_w) + zp_w, -128, 127)` . Store biases as int32: `b_q = round(b / (scale_in * scale_w))` .

2.2 — Build the NumPy bit-exact simulator

It must replicate *exactly* what C will do — same operation order, same rounding:

- int8 weights (symmetric), int8 activations; MACs in int64, accumulator kept as int32
- requantize: fixed-point multiplier + shift, **round-half-to-even**, saturate [−128, +127]
- ReLU in the int8 domain (clip 0..127 after requantize)
- softmax: fixed-point exp per the spec (LUT or shift-based), int8 in → int8 out — write the exact algorithm in the spec doc now

2.3 — Quantized-accuracy check

Run the full test set through the simulator → quantized accuracy. Expect at most a 1–2% drop from FP32.

> **GATE 2 — end of Day 2 morning:** simulator is deterministic (same inputs → identical outputs on re-run) · quantized accuracy within ~2% of FP32 · requantize spec updated with final multiplier/shift/rounding. **Everything from here on is tested against THIS simulator.**

> **If it fails:** accuracy drop > 2% → widen the calibration set, use 99.9th-percentile clipping, and re-check zero-point arithmetic (off-by-one zero-point is the #1 PTQ bug).

STAGE 3 Day 2 — Export Model + Golden Vectors

Generate everything the C side consumes. **No C code may hardcode a shape or a scale** — everything comes from these artifacts.

3.1 — Export artifacts

Artifact	Contains	Consumed by
`models/<model>/model.json`	Layer graph, shapes, kernel sizes, quant scales/zps, per-layer buffer sizes, total arena size	C dispatcher (compiled in via a generated header)
`models/<model>/model_data.c/.h`	INT8 weights, int32 biases, all scales/zps as C arrays — `const`, flash-resident	linked into firmware
`tests/golden/<model>/`	Per-layer + full-model vectors (int8 arrays + scales + expected int8 outputs, tolerance 0)	host tests + ESP32 tests

Tip: store weights row-major (C order) and state it in a header comment — C and NumPy must read the same memory layout. Generate a `model_config.h` with the arena size as a #define.

3.2 — How to test the exporter (round-trip test)

Parse the generated `model_data.c` back in Python, run the simulator on the re-imported weights, and confirm outputs are identical to the original run. This catches layout/precision errors before any C runs.

> **GATE 3 — end of Day 2 afternoon:** round-trip test passes · model_data.c compiles standalone on the host · golden vectors exist for every layer of both models.

STAGE 4 Day 2–3 — Build the C Runtime + Kernels

Order matters: **host first, ESP32 second** — the same kernel code is tested on host (fast, easy debugging), then run on target. Test each kernel in isolation against golden vectors *before* wiring the dispatcher.

4.1 — Build order (each row = one commit + one test)

#	Component	How to test it
1	Static arena + buffer manager (fixed regions, no malloc)	Unit test: allocate/free patterns; assert no overlap; assert total size matches arena size from model.json
2	Platform HAL (cycle counter, timer, print)	Host: prints fake cycles. ESP32: `esp_cpu_get_cycle_count()` delta around a busy loop; cycles ≈ 240 MHz × measured wall time
3	`requantize_int32_to_int8` helper	Golden vectors from the simulator: exact bit-match on ~1000 random int32 values across the scale range
4	`fc_int8` (fully-connected / GEMM)	Per-layer golden vectors: bit-exact. Test both single-sample (N=1) and batched input
5	ReLU + fixed-point softmax (exact spec algorithm)	Golden vectors bit-exact; softmax outputs sum to ~255 within quantization resolution
6	`conv1d_int8` (with padding option)	Golden vectors bit-exact for stride 1; test pad=valid and pad=same; test k=3 and k=5; cross-check 5 random shapes against the simulator
7	Dispatcher: walks model.json, allocates buffers, calls kernels in order	Full-model golden vector: bit-exact end-to-end on host
8	Benchmark harness (N inferences, cycles, ms, RAM, flash)	Run naive vs optimized on host first (both must be bit-exact), then on ESP32

4.2 — Optimized kernels (same interface, same bit-exact rule)

After the naive versions pass, add optimized variants selected by a build flag or dispatcher config: packed weights + 4×-unrolled GEMM; im2col-free conv1d with sliding-window accumulation. **Every optimized kernel must pass the exact same golden-vector test** — optimized but wrong is worth nothing.

> **GATE 4 — end of Day 3:** all kernels bit-exact on host · full-model inference bit-exact on host for both models · firmware compiles for ESP32 · full-model inference bit-exact on ESP32 · naive vs optimized cycle counts exist for at least the MLP.

> **If it fails:** use the per-layer vectors to find the *first layer* where C diverges from the simulator, and debug only that layer (print the int32 accumulator for one output element in both). Nine times out of ten it is rounding or memory layout — not the math.

STAGE 5 Day 4 — Benchmark, Results, Demo

5.1 — Benchmark protocol (how to measure correctly)

Metric	Method	Validity check
Cycles & latency	10 warm-up runs, then median of ≥ 100 timed runs; `esp_cpu_get_cycle_count()` around `model_infer()` only; report cycles AND ms (cycles / 240 MHz)	Disable WiFi/BT; pin task to one core; two consecutive medians within ~2%
Peak RAM	Arena size (from model.json) + stack high-water mark; .bss/.data sizes from the .map file	0xAA guard band after the arena, verified intact after a run — proves no overflow
Flash	.bin/.elf size + .map sections (text + rodata = code + weights)	Weights visible in .map ≈ model size from export
Accuracy on target	Push test vectors over UART, run inference on ESP32, compare with simulator outputs	Bit-exact match → on-target accuracy = simulator accuracy automatically

Fairness rule: naive and optimized must be built with identical compiler flags (`-O3` , set globally in the build config, not per file) and run on identical inputs. Report the environment: ESP32 @ 240 MHz, ESP-IDF version, flags.

5.2 — Fill this results table (it is the deliverable)

```
Model: 1D-CNN   |   ESP32 @ 240 MHz   |   -O3   |   ESP-IDF v5.x
+--------------------+------------+------------+---------------+
| Metric             |  Naive C   |  Optimized |  Improvement  |
+--------------------+------------+------------+---------------+
| Cycles / inference |            |            |               |
| Latency (ms)       |            |            |               |
| Peak RAM (KB)      |            |            |               |
| Flash (KB)         |            |            |               |
| Accuracy INT8 (%)  |            |            |               |
| Accuracy FP32 (%)  |  reference value from training (both cols) |
+--------------------+------------+------------+---------------+
(repeat for the MLP)
```

5.3 — Final verification checklist

Check	Pass condition
Bit-exactness	All per-layer and full-model golden vectors pass on host AND on ESP32
Reproducibility	One command rebuilds everything from a clean checkout: export → build → test → benchmark
No dynamic memory	`grep -rn "malloc|calloc|free(" firmware/src` returns zero hits
Determinism	Two benchmark runs produce identical outputs (bit-exact runtime ⇒ deterministic)
Documentation	README: architecture diagram, how to run the demo, how to read the results

GATE 5 — end of Day 4: results table filled · demo runs with one command · v0.1 tagged on GitHub. **You now have a working TinyML framework.**

Debugging map — where to look when a test fails

Symptom	Check first	Then
C output ≠ golden vector (one layer)	Rounding mode (half-to-even vs half-away), layout (row-major vs column-major), zero-point handling	Print the int32 accumulator for one output element in simulator and C; diff at the first differing index
End-to-end mismatch but all layers pass alone	Dispatcher buffer aliasing (a kernel overwriting its own input), wrong buffer index	Run the dispatcher with each layer writing to distinct buffers; re-enable overlap only after it passes
ESP32 result ≠ host C result	Plain `char` signedness (use `int8_t` /explicit types everywhere), alignment, optimization-dependent undefined behavior	Build ESP32 at -O0 vs -O2; if results change, there is UB — run host build with sanitizers (`-fsanitize=address,undefined`)
Cycle counts unstable	WiFi/BT active, other FreeRTOS tasks, cache effects	Disable radio, pin task to one core, warm-up runs, report the median
Quantized accuracy drop > 2%	Zero-point arithmetic, calibration set too small or unrepresentative	Percentile clipping; plot per-layer output distributions FP32 vs INT8 side by side
ESP32 hard fault during inference	Arena overflow (a buffer larger than declared), main-task stack too small	0xAA guard band after the arena; increase main task stack in `idf.py menuconfig`

After Day 4 — backlog (deliberately deferred)

Item	Estimate	Why it is safe to defer
Port to NXP MCXN236 (Cortex-M33) + DWT cycle benchmarking	~1–2 person-weeks	All target-specific code is isolated in the platform HAL + intrinsics; kernels and runtime are portable C
CMSIS-NN baseline integration	~3–4 days	The same golden vectors from this sprint verify the CMSIS-NN kernels
Buffer-lifetime overlap planner (peak-RAM reduction)	~2–3 days	Planner only emits a new arena layout into model.json; the runtime already reads the arena size from there
Case-study application (live sensor / keyword spotting)	~1 week	v0.1 already runs the models; the app adds a new input path (mic/sensor) feeding the same runtime

One-page summary — the whole process at a glance

```
DAY 1  ▸ Stage 0: toolchains + repo + 3 contracts (requantize spec, model.json schema, golden-vector format)
       ▸ Stage 1: train MLP + 1D-CNN (FP32)                        [GATE 1: models trained + saved]

DAY 2  ▸ Stage 2: INT8 quantize + NumPy bit-exact simulator        [GATE 2: quantized acc within ~2%]
       ▸ Stage 3: export model.json + model_data.c/.h + vectors    [GATE 3: round-trip test passes]
       ▸ Stage 4a: host kernels bit-exact vs vectors               [GATE 4a: full model bit-exact on host]

DAY 3  ▸ Stage 4b: ESP32 build + bit-exact on target              [GATE 4b: bit-exact on ESP32]
       ▸ Stage 4c: optimized kernels (packed GEMM, conv1d), bit-exact
       ▸ Stage 5a: benchmark harness                               [GATE 4: cycle counts for both models]

DAY 4  ▸ Stage 5b: benchmark table + README + demo script          [GATE 5: one-command demo, tag v0.1]
```

Golden rule: never debug two unknowns at once. Every stage has a reference (what correctness looks like) and a gate (when you may move on). The reference proves the stage is correct; the gate proves the pipeline is ready for the next stage.